

ADBC Assignment

Option 1: Library Management System API

Complete Step-by-Step Guide on Ubuntu

Stack: Python · Flask · MySQL · Postman

Tasks Covered: Schema · Sample Data · 4 APIs · Postman Testing

Assignment Overview

This guide walks you through completing the Library Management System API assignment from scratch on Ubuntu. You will set up MySQL, create the database schema, insert sample data, build a Flask REST API with 4 endpoints, and test everything using Postman.

What You Will Build

- **Database:** 3 tables — Books, Students, BookIssues
- **API 1:** POST /books — Add a new book
- **API 2:** GET /books — Get all books
- **API 3:** POST /issues — Issue a book to a student
- **API 4:** GET /students/<id>/issues — Get issue history of a student

Marks Breakdown

- Task 1 – Database Schema: **25 marks**
- Task 2 – Insert Sample Data: **5 marks**
- Task 3 – API Development: **10 marks**
- Task 4 – Postman Testing: **10 marks**

Step 1: Install Required Tools

1.1 Update Ubuntu Package List

Open a terminal (Ctrl+Alt+T) and run:

```
sudo apt update
```

1.2 Install MySQL Server

```
sudo apt install mysql-server -y
```

After installation, secure the MySQL setup:

```
sudo mysql_secure_installation
```

Note: When prompted: Set a root password, remove anonymous users, disallow remote root login, remove test database — answer Y to all.

1.3 Install Python 3 and pip

```
sudo apt install python3 python3-pip python3-venv -y
```

Verify installation:

```
python3 --version  
pip3 --version
```

1.4 Install Postman

```
sudo snap install postman
```

Note: If snap is not available: download the .tar.gz from <https://www.postman.com/downloads/> and extract it.

Step 2: Project Setup

2.1 Create Project Folder

```
mkdir ~/library_api  
cd ~/library_api
```

2.2 Create and Activate Virtual Environment

```
python3 -m venv venv  
source venv/bin/activate
```

Note: You will see (venv) at the start of your terminal prompt — this means the virtual environment is active.

2.3 Install Python Libraries

```
pip install flask flask-mysqldb
```

Tip: flask-mysqldb requires libmysqlclient-dev. If you get an error, run: `sudo apt install libmysqlclient-dev -y` then retry the pip install.

2.4 Final Folder Structure

Your project should look like this when done:

```
library_api/  
├── venv/  
├── app.py  
└── schema.sql
```

Step 3: MySQL Database Setup

3.1 Login to MySQL

```
sudo mysql -u root -p
```

Enter your root password when prompted. You will see the mysql> prompt.

3.2 Create Database and User

```
CREATE DATABASE library_db;
CREATE USER 'libuser'@'localhost' IDENTIFIED BY 'lib@1234';
GRANT ALL PRIVILEGES ON library_db.* TO 'libuser'@'localhost';
FLUSH PRIVILEGES;
USE library_db;
```

Note: You can use any password instead of lib@1234 — just remember it for the Flask app config.

3.3 Task 1 — Create Tables (25 Marks)

Paste and run the following SQL to create all 3 tables:

```
-- Table 1: Books
CREATE TABLE Books (
    book_id      INT AUTO_INCREMENT PRIMARY KEY,
    title        VARCHAR(200) NOT NULL,
    author       VARCHAR(150) NOT NULL,
    isbn         VARCHAR(20) UNIQUE NOT NULL,
    genre        VARCHAR(80),
    total_copies INT DEFAULT 1,
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table 2: Students
CREATE TABLE Students (
    student_id   INT AUTO_INCREMENT PRIMARY KEY,
    name         VARCHAR(150) NOT NULL,
    email        VARCHAR(150) UNIQUE NOT NULL,
    department   VARCHAR(100),
    phone        VARCHAR(15),
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table 3: BookIssues
CREATE TABLE BookIssues (
    issue_id     INT AUTO_INCREMENT PRIMARY KEY,
    book_id      INT NOT NULL,
    student_id   INT NOT NULL,
    issue_date   DATE NOT NULL,
    return_date  DATE,
    status       ENUM('issued', 'returned') DEFAULT 'issued',
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (student_id) REFERENCES Students(student_id)
);
```

Verify tables were created:

```
SHOW TABLES;
```

Expected output:

```
+-----+
| Tables_in_library_db |
+-----+
| BookIssues           |
| Books                |
| Students             |
+-----+
```

Step 4: Task 2 — Insert Sample Data (5 Marks)

Still inside the MySQL prompt, run the following INSERT statements:

4.1 Insert 5 Books

```
INSERT INTO Books (title, author, isbn, genre, total_copies) VALUES
('Python Crash Course',      'Eric Matthes',      '978-1593276034', 'Programming', 3),
('Clean Code',               'Robert C. Martin', '978-0132350884', 'Programming', 2),
('Database Systems',         'Ramez Elmasri',     '978-0133970777', 'Database',    4),
('Artificial Intelligence',   'Stuart Russell',    '978-0136042594', 'AI',          2),
('Computer Networks',        'Andrew Tanenbaum',  '978-0132126953', 'Networking',  3);
```

4.2 Insert 5 Students

```
INSERT INTO Students (name, email, department, phone) VALUES
('Rushi Chintanpalli', 'rushichintanpalli@mit.edu', 'MCA Data Science', '9876500001'),
('Priya Sharma',       'priyasharma@mit.edu',     'MCA',              '9876500002'),
('Amit Patil',          'amitpatil@mit.edu',       'BCA',              '9876500003'),
('Sneha Desai',         'snehadesai@mit.edu',      'MSc CS',           '9876500004'),
('Rohan More',          'rohanmore@mit.edu',       'BCA',              '9876500005');
```

4.3 Insert 10 Issue Records

```
INSERT INTO BookIssues (book_id, student_id, issue_date, return_date, status) VALUES
(1, 1, '2024-01-10', '2024-01-20', 'returned'),
(2, 1, '2024-02-05', NULL,         'issued'),
(3, 2, '2024-01-15', '2024-01-25', 'returned'),
(1, 3, '2024-02-01', NULL,         'issued'),
(4, 2, '2024-02-10', '2024-02-20', 'returned'),
(5, 4, '2024-01-20', '2024-01-30', 'returned'),
(2, 5, '2024-02-12', NULL,         'issued'),
(3, 3, '2024-03-01', NULL,         'issued'),
(4, 4, '2024-03-05', '2024-03-15', 'returned'),
(5, 5, '2024-03-10', NULL,         'issued');
```

Exit MySQL prompt:

```
EXIT;
```

Step 5: Task 3 — Flask API Development (10 Marks)

Navigate to your project folder and create `app.py`:

```
cd ~/library_api
nano app.py
```

Paste the complete code below into nano. Save with Ctrl+O, Enter, then Ctrl+X to exit.

Complete app.py Code

[illegible]

[illegible]

Step 6: Run the Flask Server

Make sure your virtual environment is active, then run:

```
cd ~/library_api
source venv/bin/activate
python3 app.py
```

You should see output like:

```
* Running on http://127.0.0.1:5000
* Debug mode: on
```

Note: Keep this terminal open while testing with Postman. The server must be running.

Tip: If you see 'Address already in use' error, another process is using port 5000. Kill it with: `sudo fuser -k 5000/tcp` then run `python3 app.py` again.

Step 7: Task 4 — Postman Testing (10 Marks)

Open Postman. For each API below, create a new request, set the method, URL, body (if needed), and click Send.

API 1 — Add Book (POST /books)

Method: POST

URL: http://127.0.0.1:5000/books

Body: Select raw → JSON, paste:

```
{
  "title": "Data Science Handbook",
  "author": "Jake VanderPlas",
  "isbn": "978-1491912058",
  "genre": "Data Science",
  "total_copies": 2
}
```

Expected Response (201):

```
{
  "book_id": 6,
  "message": "Book added successfully"
}
```

API 2 — Get All Books (GET /books)

Method: GET

URL: http://127.0.0.1:5000/books

No Body needed.

Expected Response (200):

```
{
  "books": [ {...}, {...}, ... ],
  "count": 6
}
```

API 3 — Issue Book (POST /issues)

Method: POST

URL: http://127.0.0.1:5000/issues

Body: raw → JSON:

```
{
  "book_id": 1,
  "student_id": 2,
  "issue_date": "2024-04-01"
}
```

Expected Response (201):

```
{
  "issue_id": 11,
  "message": "Book issued successfully"
}
```

API 4 — Student Issue History (GET /students/1/issues)

Method: GET

URL: http://127.0.0.1:5000/students/1/issues

No Body needed. (Change 1 to any student_id)

Expected Response (200):

```
{
  "count": 2,
  "issues": [
    {
      "author": "Eric Matthes",
      "isbn": "978-1593276034",
      "issue_date": "...",
      "return_date": null,
      "status": "issued",
      "title": "Python Crash Course"
    },
    ...
  ],
  "student_id": 1
}
```

Note: Take a screenshot of each Postman response showing the URL, request body, and response body. Include these screenshots in your assignment submission.

Step 8: Common Errors and Fixes

Error: ModuleNotFoundError: No module named 'flask_mysqlldb'

Virtual environment might not be active. Run:

```
source ~/library_api/venv/bin/activate
pip install flask flask-mysqldb
```

Error: Unable to connect to MySQL

Check MySQL service is running:

```
sudo systemctl start mysql
sudo systemctl status mysql
```

Error: 1045 Access denied for user 'libuser'

Re-create the MySQL user with correct password:

```
sudo mysql -u root -p
GRANT ALL PRIVILEGES ON library_db.* TO 'libuser'@'localhost' IDENTIFIED BY 'lib@1234';
FLUSH PRIVILEGES;
EXIT;
```

Error: ImportError: libmysqlclient.so not found

```
sudo apt install libmysqlclient-dev -y
pip install flask-mysqldb
```

Postman: Could not get response / Connection refused

Make sure Flask server is still running in a terminal. Check the terminal for errors.

How to see data in MySQL to verify

```
sudo mysql -u root -p
USE library_db;
SELECT * FROM Books;
SELECT * FROM Students;
SELECT * FROM BookIssues;
EXIT;
```

Quick Reference — All Commands

Full Setup (Run Once)

```
sudo apt update
sudo apt install mysql-server python3 python3-pip python3-venv libmysqlclient-dev -y
sudo snap install postman
sudo mysql_secure_installation
mkdir ~/library_api && cd ~/library_api
python3 -m venv venv
source venv/bin/activate
pip install flask flask-mysqldb
```

Every Time You Work on This Project

```
cd ~/library_api
source venv/bin/activate
python3 app.py
```

API Endpoints Summary

POST	http://127.0.0.1:5000/books	Add a book
GET	http://127.0.0.1:5000/books	Get all books
POST	http://127.0.0.1:5000/issues	Issue a book
GET	http://127.0.0.1:5000/students/1/issues	Student issue history

Good luck with your ADBC Assignment!

Option 1 — Library Management System API